

Frontend Developer Guide — Angular SPA

Read [OVERVIEW.md](#) first if you haven't. This document is the deep dive on `frontend/` — architecture, state management, the key user flows, the Ionic/Bootstrap setup, and the specific bugs we hit while building this and how they were fixed.

1. Tech stack

- **Angular 18, standalone components only** — there are no `NgModule` s in this codebase. Every component declares its own `imports: [...]` array.
- **Angular signals** for state, not `NgRx` or any other state library. See § 3.
- **Control flow**: `@if` / `@for` / `@switch` exclusively — never the old `*ngIf` / `*ngFor` structural directives.
- **Bootstrap 5** for all visible styling — buttons, forms, modals, layout utility classes, the whole purple design system that matches the original React prototype pixel-for-pixel.
- **Ionic 8** — installed, but used **structurally only** (see § 5). No visible Ionic theming anywhere; every interactive control is Bootstrap + custom SCSS.
- Build tool: Angular CLI (`ng serve` , `ng build`), package manager `npm`.

2. Directory layout

```
frontend/src/app/
├── components/
│   ├── chat-area/           Main chat surface – message list + composer host
│   ├── composer/           Message input box (auto-grow, @Oala mention, send)
│   ├── chat-progress/      "thinking..." / streaming status indicator
│   ├── chat-turn-divider/  Visual separator between conversation turns
│   ├── sidebar/            Conversation list – CRUD, session usage footer
│   ├── navbar/             Top bar – model picker, Tools menu, sign out
│   ├── protocol-renderer/  Turns the AI's JSON envelope into rendered UI
│   ├── mini-pathway/       Mini career-pathway generator + drawer
│   ├── pathway-whiteboard/ Drag-and-drop pathway editor
│   ├── objectives/        Guided-activity workspace
│   └── warehouse/         Course/provider/skills/region UI – rendered inline in chat,
see § 4
│   ├── research-details/   "Explore more" drill-down card
│   ├── clarification-questions/ Built, NOT wired in yet – see § 8
│   ├── skill-suggestions/  Empty directory – never built – see § 8
│   ├── modals/            One subfolder per modal: settings, user-profile,
│   │                     career-context, token-inspector, context-inspector,
│   │                     memory-timeline, analytics(-dashboard), benchmark,
│   │                     advanced-lab, export, location-prompt,
│   │                     clarification-questions
│   └── shared/            Reusable primitives (ui-kit controls, etc.)
├── services/
│   ├── token-lab.service.ts The central chat/conversation state store (see § 3)
│   ├── api.service.ts      Reusable HTTP/SSE wrapper (not fully adopted everywhere – see
note below)
│   ├── auth.service.ts     Login/logout/session check
│   └── objectives.service.ts Objectives workspace API + state
```

```

|   └─ routing.service.ts      Client-side routing/skill-suggestion logic (see § 8)
├─ app.component.ts/html/scss  Root shell – ion-app/ion-content wrapper, auth gate, modal
hosts
├─ app.config.ts               Bootstrap providers (router, HTTP client, Ionic)
└─ app.routes.ts

```

A pre-existing inconsistency worth knowing about: `ApiService` is a clean, reusable HTTP/SSE wrapper, but several components/services (notably `TokenLabService` itself) still reimplement fetch + header logic directly rather than going through it. Not a bug, just duplication — worth consolidating onto `ApiService` opportunistically if you're touching that code anyway, not a dedicated refactor task on its own.

3. State management

No NgRx, no third-party store. State lives in Angular **signals**, centralized mostly in `TokenLabService` (injected wherever needed):

- `activeConversation()`, `activeConversationId()`, `activeModelId()` — current conversation state.
- `locationProvided()` — whether the user has given a location (for local course/job relevance).
- Streaming state during a live turn (phase text, partial content, token usage).

Components read these signals directly in templates (`lab.activeConversation()`) and call service methods (`lab.sendMessage(...)`) to mutate state — no dispatch/reducer/effect ceremony. If you're coming from NgRx or Redux, this is intentionally simpler: one service, plain method calls, signals for reactivity.

4. The user flow, frontend-side

1. **AppComponent** gates everything on `auth.isAuthenticated`. Unauthenticated → login form. Authenticated → the full shell: `<app-navbar>` + `<chat-area>` + a location-prompt modal (first login only) + every tool modal (each self-hides via an `[open]` input bound to `activeTool()`).
2. **NavbarComponent** 's "Tools" dropdown sets `activeTool()` via `(openTool)` — this is what shows/hides each modal in `app.component.html`. If you're adding a new tool panel, this is the pattern: add a `ToolMenuKey` union member, a navbar button, and an `@if (activeTool() === 'your-key') { <app-your-modal .../> }` block.
3. **ChatAreaComponent** hosts the message list and `<app-composer>`. Each assistant message is handed to **ProtocolRendererComponent**, which switches over `content_blocks[].type` (heading/text/list/steps/table/comparison/callout/key_value/cards/timeline/flow/pathway_map/scorecard/chart/progress/checklist) and renders each block type's own sub-template. The active `interaction` (a question with select/ranked-select/form/free-text input) is rendered by a nested interaction component and its answer is submitted back through `TokenLabService`.
4. **Mini-pathway, Objectives, Whiteboard, Research** are separate components opened as overlay/drawer panels from the Tools menu or, for research, inline from a message's "Explore more" affordance (`msg.researchOffer`, populated by the backend on every turn — see `BACKEND.md § 8`).
5. **Warehouse data** (course/provider/career/local-area info) renders inline in chat the same way research does: `ChatMessage.warehouseData` (a `WarehousePack`, populated by the backend on turns where it's relevant — see `BACKEND.md § 7`) is read in `chat-area.component.html` and, when present, handed straight to three of the five warehouse components — no separate "Warehouse" tool panel exists, by design (see the doc comment at the top of `warehouse.types.ts`: these components only ever render an already-fetched pack, they never fetch data themselves):
 - `<app-warehouse-courses [pack]="msg.warehouseData">` when `.courses.length`

- `<app-warehouse-connections [pack]="msg.warehouseData">` when `.connected` is present (this one internally composes `LocalAreaGuideComponent` and `SkillsExplorerComponent` — you don't wire those two yourself)
- `<app-provider-comparison [data]="msg.warehouseData.comparison">` when a comparison was built (e.g. "which provider is better" questions)

5. Ionic — structural shell only {#ionic}

Why Ionic is here at all: it's used purely as a structural/layout shell (`ion-app` , `ion-content`) so a Capacitor-based mobile build stays viable later, without any Ionic visual theming leaking into the Bootstrap-styled UI. This was a deliberate early project decision — every visible control (button, input, select, modal, toggle) is Bootstrap + custom SCSS, never an Ionic component like `<ion-button>` .

Wiring, if you need to touch it:

- `app.config.ts` — `provideIonicAngular()` in the providers array.
- `app.component.ts` — imports `IonApp` , `IonContent` from `@ionic/angular/standalone` and uses them in its own template.
- `app.component.html` — the whole app is wrapped in `<ion-app><ion-content [scrollY]="false" [fullscreen]="true"> ... </ion-content></ion-app>` . `[scrollY]="false"` is important: it stops `ion-content` 's own internal scroll container from taking over, leaving Bootstrap's existing flex/overflow layout in full control of scrolling.
- `styles.scss` — imports exactly three Ionic CSS files, right after the Bootstrap import:

```
@import 'bootstrap/scss/bootstrap';
@import '@ionic/angular/css/core.css';
@import '@ionic/angular/css/normalize.css';
@import '@ionic/angular/css/structure.css';
```

Deliberately does NOT import `typography.css` , `display.css` , or the other optional Ionic utility CSS files — those restyle `h1` – `h6` , links, and add Ionic's own utility classes, which would visually fight Bootstrap. Don't add them without a specific reason and a visual check afterward.

- `ion-app { contain: none !important; }` in `styles.scss` — **not dead code**, even though it might look like a leftover. Ionic's `core.css` gives `.ion-page` (a class Ionic auto-applies to `ion-app`) `contain: layout size style` , which would clip any Bootstrap `position: fixed` element (like a modal) to the `ion-app` box instead of the real viewport. This rule turns that off.

The bug we hit here (worth knowing if Ionic ever "disappears" again)

`provideIonicAngular()` wires up the **JS runtime** (the custom element registry) but does **not** pull in Ionic's required global CSS on its own — Angular CLI's `angular.json` styles array only had `src/styles.scss` in it, and without the three CSS imports above, `ion-app` / `ion-content` rendered as zero-size boxes (`display: inline` , no positioning at all), which pushed the *entire application* off-screen — sidebar, chat, everything. It looked like a catastrophic layout bug but the actual root cause was one missing set of CSS imports. If you ever see the whole app apparently vanish/collapse to a blank page after touching `app.config.ts` or the Ionic wrapper, check `styles.scss` first.

6. Angular 18 gotchas we hit (save yourself the debugging time)

These bit us more than once across different components during this build — worth knowing up front:

1. `@else if (expr; as alias)` is invalid. The `as` alias binding is only legal on the *primary* `@if` block (NG5002 at build time). If you need an alias on a later branch, restructure into a nested `@if` inside the `@else`:

```
@if (a) { ... }
@else {
  @if (b; as alias) { ... }
}
```

2. **Class field initializers run before constructor-parameter properties are assigned.** `sessionStats = this.lab.sessionStats;` as a field initializer throws TS2729: Property 'lab' is used before its initialization if `lab` is a constructor-parameter property (`constructor(public lab: TokenLabService)`). Fix: declare the field bare (`sessionStats!: ...`) and assign it inside the constructor *body*, not as a field initializer.
3. **Gemini-sourced "array" fields aren't always arrays at runtime**, even when typed as arrays in TypeScript — the model's actual output doesn't always match the type. Guard with `Array.isArray(x) ? x : []` before calling `.filter()` / `.map()` /etc. on anything that ultimately comes from a Gemini response (we hit this specifically on `component.content` in a couple of objectives-related components).
4. **A shared hardcoded `localStorage` key across multiple instances of the same resizable-drawer pattern will fight itself.** `DrawerResizeController` originally hardcoded one key (`'yuzee-mini-pathway-width'`) for *all* resizable drawers, so the Objectives workspace and Mini Pathway drawers clobbered each other's persisted width. Fixed by making the width key a constructor parameter with a default, so each usage passes its own key (`'yuzee-objectives-width'` , etc.).

7. Testing & local dev workflow

```
cd frontend
npm install
npm start           # ng serve --proxy-config proxy.conf.json, localhost:4200
npm test            # Karma/Jasmine unit tests
npm run build       # production build, output to dist/
npm run build:spring # production build, copied into
../backend/src/main/resources/static/
```

`proxy.conf.json` forwards `/api/*` to `http://localhost:8080` — the backend must be running separately for anything beyond the static shell to work in dev.

A couple of dev-loop notes from this build:

- If you're testing against a long-lived browser tab across many backend restarts/hot-reloads, **don't trust its accumulated console error history** — Vite/Angular dev server HMR can leave stale errors sitting in a tab's console from minutes or hours earlier. When diagnosing "is this a real, current bug," reload fresh (or open a brand-new tab) and check the console from that clean state, rather than reading a long-lived tab's full history.
- Bundle size budgets in `angular.json` are currently 1.2MB (warning) / 2MB (error) for the initial bundle — raised once already as real features were added; if you hit the warning again from legitimate new code, that's the knob to adjust, not a sign something's wrong.

8. Known gaps — built but not wired in

These are real, complete pieces of frontend code that currently render nothing to a user because nothing imports them. Not something to "just fix" without checking scope first — each one implies a corresponding backend wiring decision too (see `BACKEND.md` § 5 overview gaps):

- `components/warehouse/*` ~~not wired in~~ — **fixed:** `warehouse-courses` , `warehouse-connections` and `provider-comparison` are now imported into `chat-area.component.ts` and render inline against real backend data — see § 4 above.
- `ClarificationQuestionsModalComponent` (wraps a fully-built `ClarificationQuestionsCardComponent`) — not imported anywhere. In the original prototype this was rendered globally alongside the other modals.
- `ObjectiveSuggestionsComponent` — a client-side keyword-match replacement for a dropped embedding-based suggestion pipeline (there's a doc comment in `objectives.service.ts` explaining the substitution). Built, never imported.
- `SkillSuggestionsComponent` — `components/skill-suggestions/` is an empty directory. This one was never built at all, unlike the others above which exist but are unwired.
- A handful of shared UI-kit primitives (`RangeSliderComponent` , `SearchableSelectComponent` , `ToggleSwitchComponent` , `SegmentedControlComponent` under `components/shared/ui-kit/`) are built but used nowhere outside their own files — the settings surfaces that would have used them currently fall back to plain HTML controls instead.

9. Live-testing checklist (what to click through after a change)

If you've touched anything shell-level (`app.component.*` , `styles.scss` , `app.config.ts`), do a quick manual pass before calling it done — automated tests don't cover visual layout regressions:

1. Login screen renders correctly, centered, no console errors on a **fresh** tab load.
2. Location-prompt modal appears and can be skipped/confirmed.
3. Send a chat message, confirm streaming status text appears, then the rendered protocol response (not raw JSON — see `BACKEND.md` § 4 for when raw JSON is expected/acceptable).
4. Open at least one Tools-menu modal and confirm it overlays correctly with no double scrollbars.
5. Resize the mini-pathway and/or objectives drawer and confirm the width persists across a reload (and that the two drawers don't clobber each other's width — see § 6.4).